Laboratorio 3 – Herencia

Sistema de órdenes de servicio TecniHogar

Manuel Rio · Carné 16639

18 de octubre de 2026

1. Requisitos funcionales

1.1 Información que se conserva

Entidad	Datos	Restricciones
Orden	Código, cliente, dirección, horas estimadas, zona, estado, visitas realizadas, monto de registro y monto por visitas adicionales	Código no vacío y único; textos no vacíos; horas > 0
Diagnóstico	-------	Horas ≤ 1.5
Instalación	Cantidad de puntos	Entero > 0
Inst. eléctrica certificada	Puntos y valor declarado de materiales	Puntos > 0; materiales > 0
Mantenimiento A/C	Capacidad en BTU	9,000–60,000; solo zona metropolitana
Cuadrilla	Código, capacidad de horas, certificación en refrigeración	Código único; capacidad > 0
Jornada	Código, cuadrilla, órdenes, órdenes con resultado, estado	Una jornada sin cerrar por cuadrilla

1.2 Operaciones y condiciones de validez

Toda operación inválida se rechaza con un mensaje y sin modificar el estado: se valida antes de cambiar cualquier dato.

Operación	Efecto	Condiciones de validez
Cotizar	Muestra costo y días hábiles; no registra nada	Datos válidos según la categoría
Registrar orden	Guarda la orden PENDIENTE y cobra el monto completo	Cliente acepta; código no repetido
Registrar cuadrilla	Agrega la cuadrilla	Código no repetido; capacidad > 0
Crear jornada	Jornada EN_PREPARACION para una cuadrilla	Cuadrilla existe y no tiene jornada sin cerrar
Asignar orden	Orden pasa a ASIGNADA	Jornada en preparación; orden PENDIENTE; horas asignadas + horas estimadas ≤ capacidad; categoría compatible con la cuadrilla
Retirar orden	Orden vuelve a PENDIENTE	Jornada en preparación; la orden pertenece a ella
Iniciar jornada	Órdenes EN_ATENCION; suma visita y Q40 desde la 2.ª	Jornada en preparación y con órdenes
Registrar resultado	Completada, pendiente o cancelada; cierra la jornada con el último resultado	Jornada en curso; orden en atención y sin resultado
Consultas y reportes	Órdenes, cuadrillas, conteo por categoría/estado, ingresos, jornadas abiertas	-------

1.3 Reglas por categoría

Categoría	Costo	Días (metro / perif.)	Visitas	Cuadrilla
Diagnóstico	Q150 / Q225 fijo	1 / 2	3	Cualquiera
Instalación	Q100 / Q175 + Q90 × ⌈máx(horas, puntos × 0.75)⌉	3 / 5	3	Cualquiera
Inst. eléctrica certificada	Instalación + Q120 + 5 % de materiales	5 / 7	2	Cualquiera
Mantenimiento A/C	Q100 + Q110 × horas + Q80	2	1	Certificada en refrigeración

2. Decisiones que dependen de la categoría

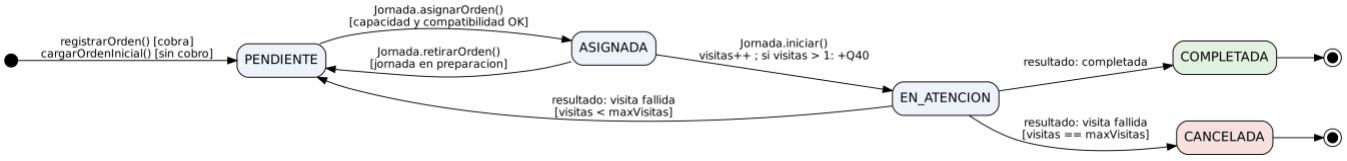
Cada decisión es un método de OrdenServicio que la subclase implementa o sobrescribe; Jornada y TecniHogar solo lo llaman.

Decisión	Método	Clase que decide
Costo	calcularCosto()	Cada subclase (la certificada usa super)
Días de atención	getDiasAtencion()	Cada subclase (la certificada: super + 2)
Visitas máximas	getMaxVisitas()	Cada subclase
Compatibilidad con cuadrilla	esCompatibleCon(Cuadrilla)	OrdenServicio (true); la sobrescribe MantenimientoAireAcondicionado
Validaciones propias	Constructores	Cada subclase, tras super(...)
Nombre en reportes	getCategoria()	Cada subclase

Por qué de ahí: la subclase tiene los datos y conoce su regla. Lo común queda una sola vez en OrdenServicio.

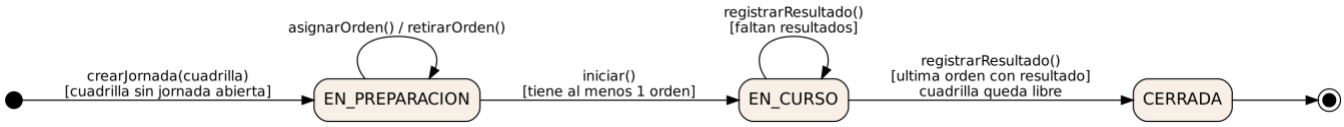
3. Estados

3.1 Orden



Una orden en otra jornada está ASIGNADA o EN_ATENCION, por eso solo se aceptan órdenes PENDIENTE (condición 3). COMPLETADA y CANCELADA son finales.

3.2 Jornada


asignarOrden() / retirarOrden()
registrarResultado()
[faltan resultados]
crearJornada(cuadrilla)
[cuadrilla sin jornada abierta]
EN_PREPARACION
iniciar()
[tiene al menos 1 orden]
EN_CURSO
registrarResultado()
[ultima orden con resultado]
cuadrilla queda libre
CERRADA

4. Clases

Visibilidad: + público, - privado, # protegido. Los atributos son privados y solo cambian por métodos que validan (estado protegido); no hay setters.

OrdenServicio

Guarda la información común de toda orden, controla su ciclo de vida (estados y visitas) y lleva sus montos cobrados. Declara las decisiones que dependen de la categoría como métodos abstractos o sobrescribibles.

Vis.	Miembro	Tipo / resultado	Parámetros	Propósito
+	CARGO_VISITA_ADICIONAL static	double = 40.0	----	Cargo por cada visita después de la primera.
-	codigo	String	----	Identificador único de la orden.
-	cliente	String	----	Nombre del cliente.
-	direccion	String	---	Dirección de la visita.
-	horasEstimadas	double	----	Horas de trabajo estimadas; se usan para la capacidad de la cuadrilla.
-	zona	Zona	----	Metropolitana o periférica.
-	estado	EstadoOrden	----	Estado actual (pendiente, asignada, en atención, completada, cancelada).
-	visitasRealizadas	int	----	Visitas iniciadas hasta ahora.
-	montoRegistro	double	----	Monto cobrado al registrar la orden (0 si es precargada).
-	montoVisitasAdicionales	double	----	Suma de cargos de Q40 por visitas adicionales.
+	OrdenServicio() constructor	----	codigo: String, cliente: String, direccion: String, horasEstimadas: double, zona: Zona	Valida los datos comunes e inicia la orden en PENDIENTE, sin visitas ni montos.
+	calcularCosto() abstracto	double	----	Costo del servicio según la categoría.
+	getDiasAtencion() abstracto	int	----	Días hábiles estimados para la visita.
+	getMaxVisitas() abstracto	int	-----	Visitas permitidas antes de cancelar.
+	getCategoria() abstracto	String	-----	Nombre de la categoría para consultas y reportes.
+	esCompatibleCon()	boolean	cuadrilla: Cuadrilla	Si la cuadrilla puede atender la orden. Por defecto true.

Vis.	Miembro	Tipo / resultado	Parámetros	Propósito
+	getDetalleCategoria()	String	-----	Datos propios de la categoría para mostrar. Por defecto vacío.
#	getTarifaBase()	double	-----	Q100 metropolitana / Q175 periférica.
+	cobrarRegistro()	void	------	Registra montoRegistro = calcularCosto() al aceptar la cotización.
+	marcarAsignada()	void	-----	PENDIENTE → ASIGNADA (rechaza otro estado).
+	marcarRetirada()	void	-----	ASIGNADA → PENDIENTE.
+	iniciarVisita()	void	-----	ASIGNADA → EN_ATENCION; suma la visita y, si no es la primera, el cargo de Q40.
+	registrarResultado()	void	completada: boolean	EN_ATENCION → COMPLETADA, PENDIENTE o CANCELADA según getMaxVisitas().
+	getCodigo / getCliente / getDireccion()	String	-----	Consultan los datos de la orden.
+	getHorasEstimadas()	double	-----	Consulta las horas estimadas.
+	getZona()	Zona	-----	Consulta la zona.
+	getEstado()	EstadoOrden	----	Consulta el estado.
+	getVisitasRealizadas()	int	-----	Consulta las visitas.
+	getMontoRegistro / getMontoVisitasAdicionales / getIngresoTotal()	double	-----	Consultan los ingresos de la orden.
+	toString() sobrescrito	String	------	Línea de consulta de la orden.

Diagnostico

Revisión previa. Precio fijo por zona; valida el máximo de 1.5 horas.

Vis.	Miembro	Tipo / resultado	Parámetros	Propósito
+	HORAS_MAXIMAS static	double = 1.5	-----	Límite de horas del diagnóstico.
+	Diagnostico() constructor	------	codigo: String, cliente: String, direccion: String, horasEstimadas: double, zona: Zona	Llama a super(...) y rechaza más de 1.5 h.

Vis.	Miembro	Tipo / resultado	Parámetros	Propósito
+	calcularCosto() sobrescrito	double	-----	Q150 metropolitana / Q225 periférica.
+	getDiasAtencion() sobrescrito	int	------	1 metropolitana / 2 periférica.
+	getMaxVisitas() sobrescrito	int	-----	3.
+	getCategoria() sobrescrito	String	----	"Diagnostico".

Instalacion

Instalación por puntos. Calcula horas facturables y cobra la fracción como hora completa.

Vis.	Miembro	Tipo / resultado	Parámetros	Propósito
+	HORAS_POR_PUNTO static	double = 0.75	----	Horas por cada punto.
+	TARIFA_POR_HORA static	double = 90.0	----	Cobro por hora facturable o fracción.
-	cantidadPuntos	int	----	Puntos a instalar (entero positivo).
+	Instalacion() constructor	----	codigo: String, cliente: String, direccion: String, horasEstimadas: double, zona: Zona, cantidadPuntos: int	Llama a super(...) y valida puntos > 0.
+	getCantidadPuntos()	int	----	Consulta los puntos.
+	getHorasPorPuntos()	double	---	cantidadPuntos × 0.75.
+	getHorasFacturables()	double	-----	max(horasEstimadas, horasPorPuntos).
+	calcularCosto() sobrescrito	double	-----	getTarifaBase() + 90 × ceil(horasFacturables).
+	getDiasAtencion() sobrescrito	int	-----	3 metropolitana / 5 periférica.
+	getMaxVisitas() sobrescrito	int	----	3.
+	getCategoria() sobrescrito	String	----	"Instalacion".
+	getDetalleCategoria() sobrescrito	String	-----	Puntos, horas por puntos y horas facturadas.

InstalacionElectricaCertificada

Es una instalación con certificado y garantía. Reutiliza el costo y los días de Instalacion mediante super y solo agrega lo propio.

Vis.	Miembro	Tipo / resultado	Parámetros	Propósito
+	CARGO_CERTIFICADO static	double = 120.0	---	Costo del certificado.
+	PORCENTAJE_GARANTIA static	double = 0.05	---	Garantía: 5 % de los materiales.
+	DIAS_EXTRA static	int = 2	-----	Días adicionales a una instalación normal.
-	valorMateriales	double	-----	Valor declarado de los materiales (> 0).
+	InstalacionElectricaCertificada() constructor	----	codigo: String, cliente: String, direccion: String, horasEstimadas: double, zona: Zona, cantidadPuntos: int, valorMateriales: double	Llama a super(...) de Instalacion y valida materiales > 0.
+	getValorMateriales()	double	-----	Consulta el valor de materiales.
+	getGarantia()	double	---	valorMateriales × 0.05.
+	calcularCosto() sobrescrito	double	----	super.calcularCosto() + 120 + garantía.
+	getDiasAtencion() sobrescrito	int	-----	super.getDiasAtencion() + 2.
+	getMaxVisitas() sobrescrito	int	---	2.
+	getCategoria() sobrescrito	String	---	"Instalacion electrica certificada".
+	getDetalleCategoria() sobrescrito	String	---	Detalle de Instalacion + materiales, certificado y garantía.

MantenimientoAireAcondicionado

Mantenimiento de A/C. Solo zona metropolitana, BTU entre 9,000 y 60,000, una sola visita y solo cuadrillas certificadas en refrigeración.

Vis.	Miembro	Tipo / resultado	Parámetros	Propósito
+	BTU_MINIMO static	int = 9000	-----	Capacidad mínima permitida.
+	BTU_MAXIMO static	int = 60000	----	Capacidad máxima permitida.
+	TARIFA_POR_HORA static	double = 110.0	------	Cobro por hora estimada.
+	CARGO_GAS static	double = 80.0	-----	Cargo fijo por gas refrigerante.
-	capacidadBTU	int	----	Capacidad del equipo.

Vis.	Miembro	Tipo / resultado	Parámetros	Propósito
+	MantenimientoAireAcondicionado() constructor	----	codigo: String, cliente: String, direccion: String, horasEstimadas: double, zona: Zona, capacidadBTU: int	Llama a super(...); rechaza zona periférica y BTU fuera de rango.
+	getCapacidadBTU()	int	-----	Consulta la capacidad.
+	calcularCosto() sobrescrito	double	------	getTarifaBase() + 110 × horasEstimadas + 80.
+	getDiasAtencion() sobrescrito	int	------	Siempre 2.
+	getMaxVisitas() sobrescrito	int	----	1.
+	getCategoria() sobrescrito	String	----	"Mantenimiento de aire acondicionado".
+	esCompatibleCon() sobrescrito	boolean	cuadrilla: Cuadrilla	true solo si la cuadrilla está certificada en refrigeración.
+	getDetalleCategoria() sobrescrito	String	-----	BTU y cargo de gas.

Cuadrilla

Datos de un grupo de técnicos: código, capacidad de horas por jornada y certificación en refrigeración.

Vis.	Miembro	Tipo / resultado	Parámetros	Propósito
-	codigo	String	----	Identificador único.
-	capacidadHoras	double	----	Horas máximas por jornada (> 0).
-	certificadaRefrigeracion	boolean	---	Si puede atender mantenimientos de A/C.
+	Cuadrilla() constructor	---	codigo: String, capacidadHoras: double, certificadaRefrigeracion: boolean	Valida código no vacío y capacidad > 0.
+	getCodigo()	String	----	Consulta el código.
+	getCapacidadHoras()	double	------	Consulta la capacidad.
+	isCertificadaRefrigeracion()	boolean	---	Consulta la certificación.
+	toString() sobrescrito	String	----	Línea de consulta.

Jornada

Agrupa las órdenes de una cuadrilla. Valida las tres condiciones de asignación, controla su propio estado y se cierra cuando todas sus órdenes tienen resultado.

Vis.	Miembro	Tipo / resultado	Parámetros	Propósito
-	codigo	String	----	Identificador (J1, J2, …).
-	cuadrilla	Cuadrilla	----	Cuadrilla que trabaja la jornada.
-	ordenes	ArrayList<OrdenServicio>	----	Órdenes asignadas.
-	ordenesConResultado	ArrayList<OrdenServicio>	----	Órdenes que ya tienen resultado en esta jornada.
-	estado	EstadoJornada	------	EN_PREPARACION, EN_CURSO o CERRADA.
+	Jornada() constructor	----	codigo: String, cuadrilla: Cuadrilla	Crea la jornada vacía EN_PREPARACION.
+	asignarOrden()	void	orden: OrdenServicio	Valida estado, que la orden esté pendiente, capacidad y esCompatibleCon; luego la asigna.
+	retirarOrden()	Void	orden: OrdenServicio	Solo en preparación; la orden vuelve a PENDIENTE.
+	iniciar()	void	-----	Rechaza jornada vacía; llama iniciarVisita() en cada orden.
+	registrarResultado()	Void	orden: OrdenServicio, completada: boolean	Registra el resultado y cierra la jornada si ya no faltan.
+	getHorasAsignadas()	double	-----	Suma de horas estimadas asignadas.
+	getCapacidadDisponible()	double	-----	capacidad − horas asignadas.
+	estaAbierta()	boolean	-----	true si no está CERRADA.
+	tieneResultado()	boolean	orden: OrdenServicio	Si la orden ya tiene resultado aquí.
+	getCodigo()	String	-----	Consulta el código.
+	getCuadrilla()	Cuadrilla	----	Consulta la cuadrilla.
+	getEstado()	EstadoJornada	------	Consulta el estado.
+	getOrdenes()	ArrayList<OrdenServicio>	------	Copia de la lista (protege el estado).
+	toString() sobrescrito	String	-------	Línea para el reporte de jornadas.

TecniHogar

Registro central de la empresa: guarda órdenes, cuadrillas y jornadas, garantiza códigos únicos y una sola jornada abierta por cuadrilla, y genera los reportes.

Vis.	Miembro	Tipo / resultado	Parámetros	Propósito
-	ordenes	ArrayList<OrdenServicio>	-----	Órdenes registradas.
-	cuadrillas	ArrayList<Cuadrilla>	------	Cuadrillas registradas.
-	jornadas	ArrayList<Jornada>	------	Todas las jornadas (abiertas y cerradas).
-	contadorJornadas	Int	-----	Genera códigos J1, J2, …
+	TecniHogar() constructor	-----	------	Inicia las listas vacías.
+	registrarOrden()	Void	orden: OrdenServicio	Valida código único, agrega y cobra el registro.
+	cargarOrdenInicial()	void	orden: OrdenServicio	Agrega sin cobrar (datos iniciales).
+	validarCodigoOrdenNuevo()	void	codigo: String	Rechaza código vacío o repetido.
+	buscarOrden()	OrdenServicio	codigo: String	Busca por código (null si no existe).
-	obtenerOrden()	OrdenServicio	codigo: String	Busca o lanza error.
+	registrarCuadrilla()	void	cuadrilla: Cuadrilla	Rechaza código repetido.
+	buscarCuadrilla()	Cuadrilla	codigo: String	Busca por código.
+	buscarJornadaAbierta()	Jornada	cuadrilla: Cuadrilla	Jornada sin cerrar de la cuadrilla o null.
+	crearJornada()	Jornada	codigoCuadrilla: String	Rechaza si la cuadrilla ya tiene jornada sin cerrar.
+	buscarJornada()	Jornada	codigo: String	Busca por código.
-	obtenerJornada()	Jornada	codigo: String	Busca o lanza error.
+	asignarOrden / retirarOrden()	void	codigoJornada: String, codigoOrden: String	Delegan en la Jornada.
+	iniciarJornada()	void	codigoJornada: String	Delega en Jornada.iniciar().
+	registrarResultado()	void	codigoJornada: String, codigoOrden: String, completada: boolean	Delega en la Jornada.
+	getIngresosPorRegistro / getIngresosPorVisitas / getIngresoTotal()	double	-----	Suman los montos de las órdenes.
+	reporteOrdenesPorCategoriaYEstado()	String	--------	Conteo por getCategoria() y estado.
+	reporteIngresos()	String	------	Ingreso total, por tipo y por categoría.
+	reporteJornadasAbiertas()	String	-----	Jornadas sin cerrar con horas y capacidad.
+	getOrdenes / getCuadrillas / getJornadas()	ArrayList	----	Copias de las listas.

Main

Interfaz de consola: muestra el menú, lee datos sin terminar ante formatos inválidos, carga los datos iniciales y muestra los mensajes de error. No contiene reglas de negocio.

Vis.	Miembro	Tipo / resultado	Parámetros	Propósito
-	scanner static	Scanner	----	Lectura del teclado.
-	empresa static	TecniHogar	---	Datos del sistema en memoria.
+	main() static	void	args: String[]	Carga datos iniciales y repite el menú hasta salir.
-	cotizarYRegistrarOrden() static	void	----	Pide datos, crea la subclase elegida, muestra costo y días, registra si el cliente acepta.
-	registrarCuadrilla / crearJornada / asignarOrden / retirarOrden / iniciarJornada / registrarResultado() static	void	----	Opciones del menú.
-	consultarOrdenes / consultarCuadrillas / mostrarOrdenesPendientes() static	void	-----	Consultas.
-	leerTexto / leerEntero / leerDecimal / leerOpcion / leerZona() static	String / int / double / Zona	mensaje: String …	Lectura segura: repite si el formato es inválido.
-	cargarDatosIniciales() static	void	----	8 órdenes pendientes y 3 cuadrillas, sin ingresos.

Constructores

Cada subclase llama primero a super(...): OrdenServicio valida los datos comunes e inicia en PENDIENTE con 0 visitas y montos en 0; luego la subclase valida lo suyo. La certificada encadena tres niveles (certificada → Instalacion → OrdenServicio). Si algo es inválido se lanza una excepción y el objeto nunca se guarda.

5. Instalación eléctrica certificada

Hereda de instalacion porque es una instalación: mismos datos y mismo cálculo, más certificado y garantía. Usa super.calcularCosto() + 120 + garantía y super.getDiasAtencion() + 2, así la fórmula de puntos y redondeo existe una sola vez.

Alternativa	Ventaja	Desventaja
Heredar de OrdenServicio	Independiente de Instalación	Duplica puntos, horas facturables y redondeo
Composición (contiene una Instalacion)	Jerarquía más plana	Hay que reenviar cada método y repetir datos
boolean certificada en Instalación	Una clase menos	Llena la clase de if, justo lo que evita el polimorfismo

6. Agregar una nueva categoría

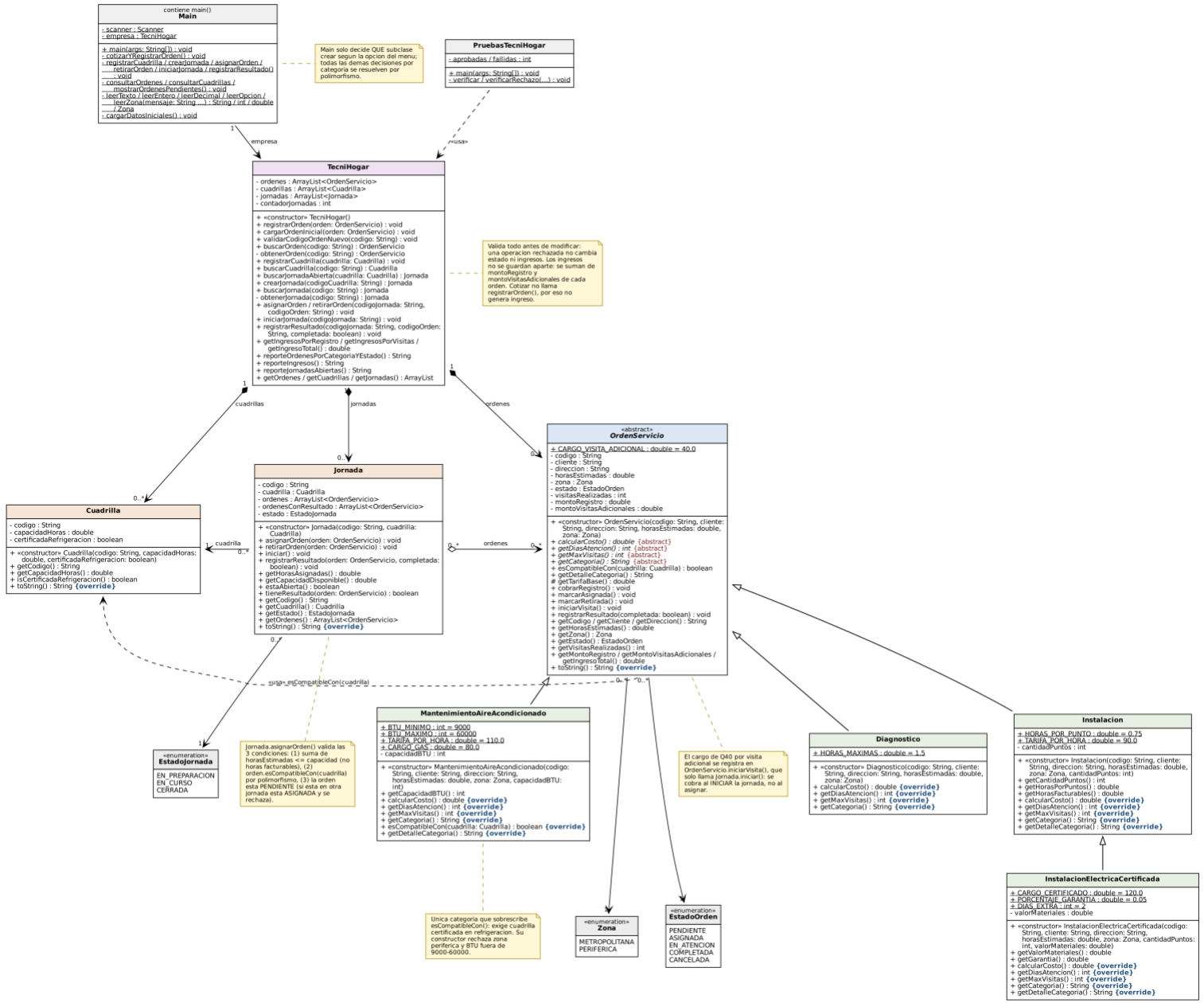
Ej.: paneles solares. Se crea una subclase de OrdenServicio con sus métodos de costo, días, visitas y, si aplica, esCompatibleCon(). Se modifica solo Main y Cuadrilla únicamente si exige una certificación nueva. Jornadas y reportes no cambian: usan los métodos polimórficos y getCategoria().

7. Diagrama UML

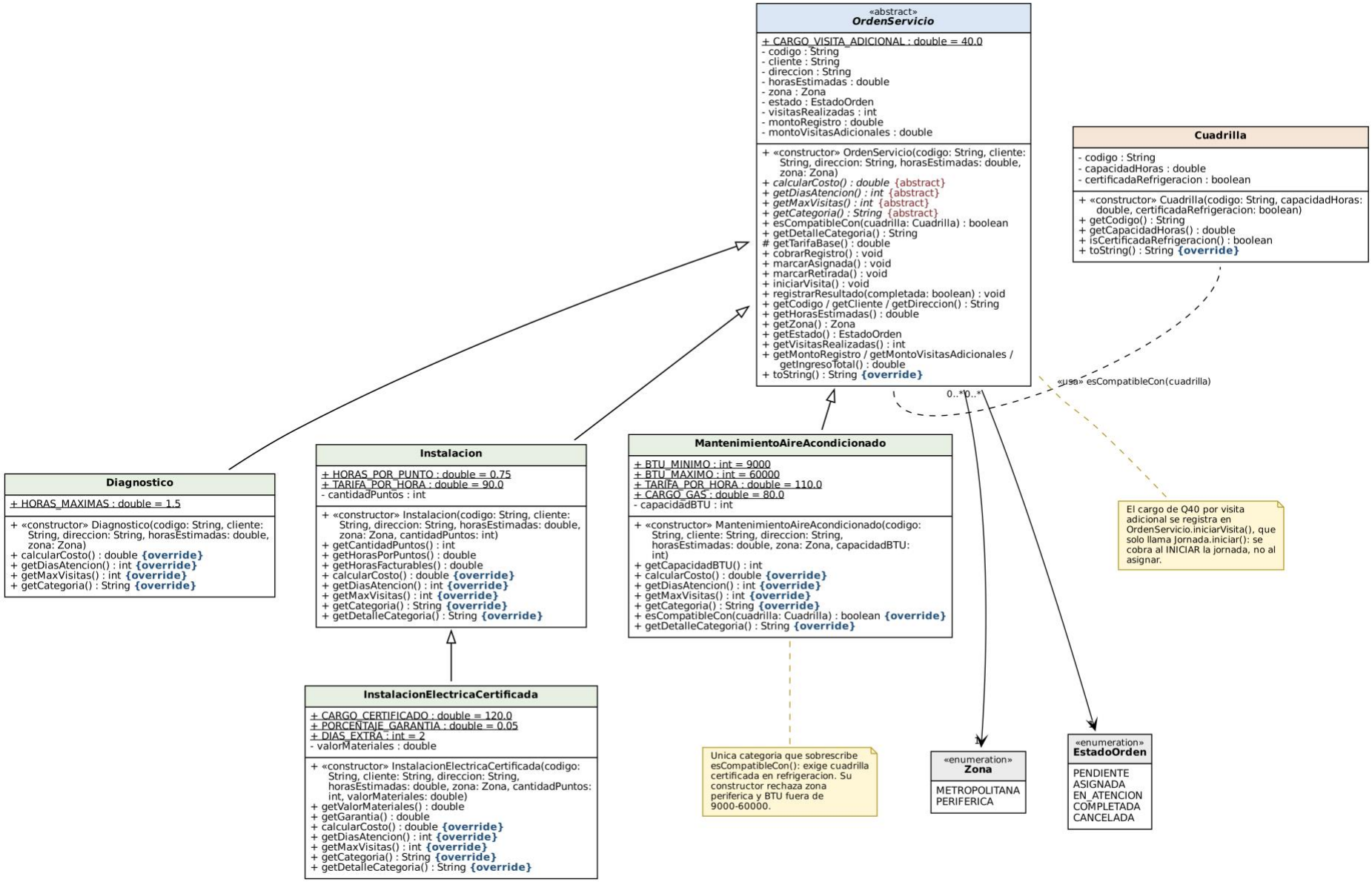
Herencia: triángulo vacío; composición: rombo lleno; agregación: rombo vacío; dependencia: punteada. Abstractos en cursiva, sobrescritos con {override}, static subrayado. Main contiene el método main.

- **Compatibilidad con la cuadrilla:** se valida en Jornada.asignarOrden() llamando a orden.esCompatibleCon(cuadrilla).
- **Cargo de Q40:** se registra en OrdenServicio.iniciarVisita(), llamado solo por Jornada.iniciar().
- **Ingresos:** TecniHogar los suma desde los montos de cada orden; cotizar no llama registrarOrden().

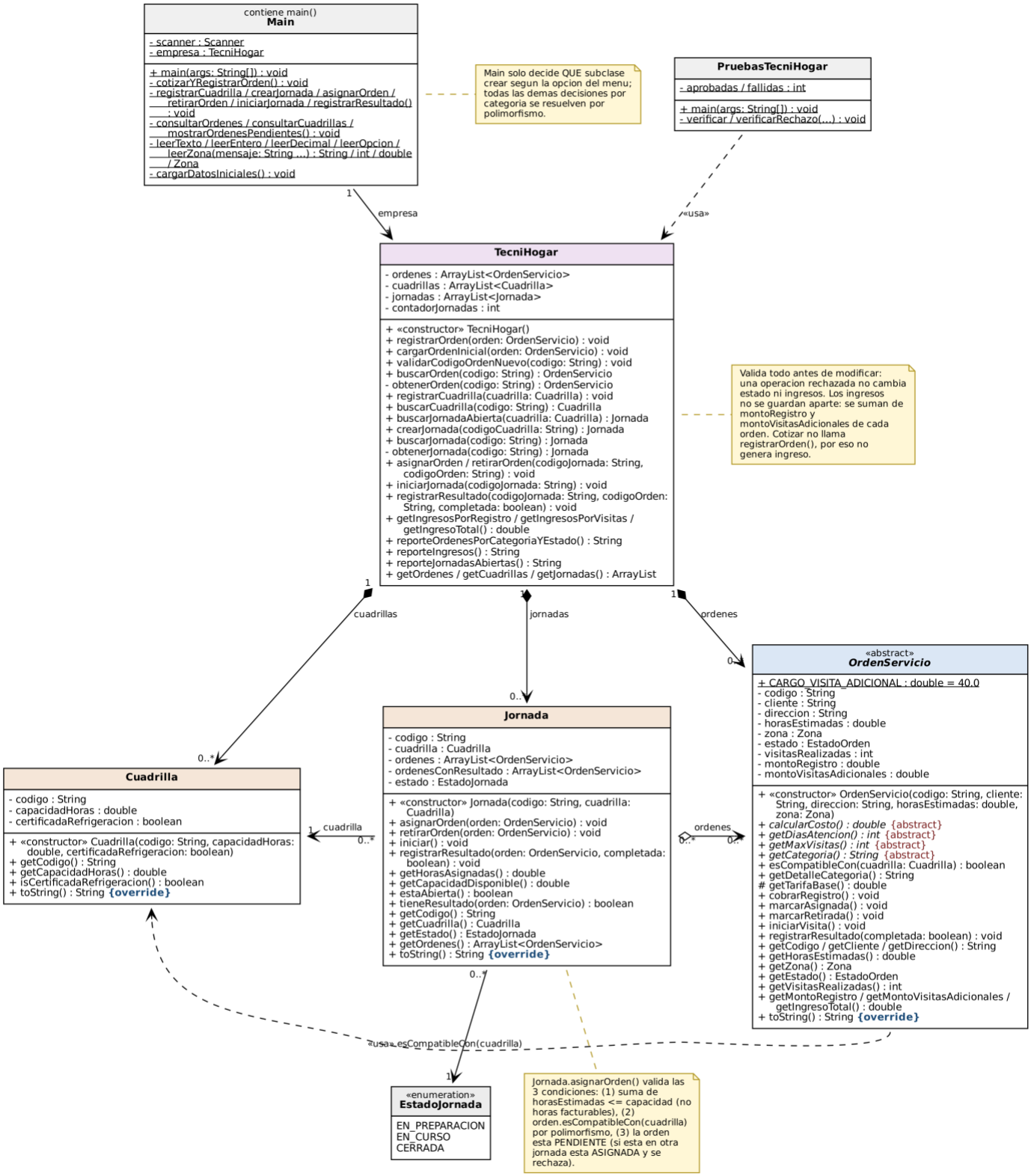
7.2 Diagrama de clases UML completo – TecniHogar



7.3 Detalle A – Jerarquía de órdenes de servicio



7.4 Detalle B – Gestión: TecniHogar, Jornada, Cuadrilla y Main



8. Pruebas

Resultados obtenidos al ejecutar java PruebasTecniHogar (60 casos en total, todos aprobados). Se muestran los representativos de cada requisito.

Caso / entrada	Esperado	Obtenido	
Instalacion: horas por puntos > estimadas (ejemplo del enunciado) 2.5 h, 5 puntos, metropolitana	Q460.00	Q460.00	**PASA**
Instalacion: estimadas > puntos y hora por fraccion (3.2 -> 4) 3.2 h, 2 puntos, periferica	Q535.00	Q535.00	**PASA**
Certificada: instalacion + Q120 + garantia 5% (ejemplo del enunciado) 2.5 h, 5 puntos, Q2000, metropolitana	Q680.00	Q680.00	**PASA**
Certificada periferica (tarifa base Q175) 1.0 h, 2 puntos, Q3000, periferica	Q625.00	Q625.00	**PASA**
Mantenimiento A/C: base + Q110/h + Q80 gas 2.0 h, 12000 BTU, metropolitana	Q400.00	Q400.00	**PASA**
Mantenimiento A/C con horas decimales (no se redondea) 1.5 h, 36000 BTU, metropolitana	Q345.00	Q345.00	**PASA**
Dias diagnostico metropolitano metropolitana	1 dia(s)	1 dia(s)	**PASA**
Dias diagnostico periferico periferica	2 dia(s)	2 dia(s)	**PASA**
Dias instalacion metropolitana metropolitana	3 dia(s)	3 dia(s)	**PASA**
Dias instalacion periferica periferica	5 dia(s)	5 dia(s)	**PASA**
Dias certificada metropolitana (3 + 2) metropolitana	5 dia(s)	5 dia(s)	**PASA**
Dias certificada periferica (5 + 2) periferica	7 dia(s)	7 dia(s)	**PASA**
Dias mantenimiento A/C (siempre 2) metropolitana	2 dia(s)	2 dia(s)	**PASA**
Diagnostico de mas de 1.5 horas 2.0 h	Rechazada	Rechazada: Un diagnostico no puede superar 1.5 horas.	**PASA**

Caso / entrada	Esperado	Obtenido	
Mantenimiento A/C en zona periferica periferica	Rechazada	Rechazada: Solo se aceptan mantenimientos de aire acondicionado en la zona metropolitana.	**PASA**
Mantenimiento A/C con BTU bajo el rango 8000 BTU	Rechazada	Rechazada: La capacidad del equipo debe estar entre 9000 y 60000 BTU.	**PASA**
Codigo de orden repetido D-1 ya existe	Rechazada	Rechazada: Ya existe una orden con el codigo D-1.	**PASA**

Pruebas aprobadas: las pruebas en general demuestran que cotizar, retirar y las operaciones rechazadas no cambian los ingresos.